

PSRemoting
Variables
Input/Output

How PS Remoting works

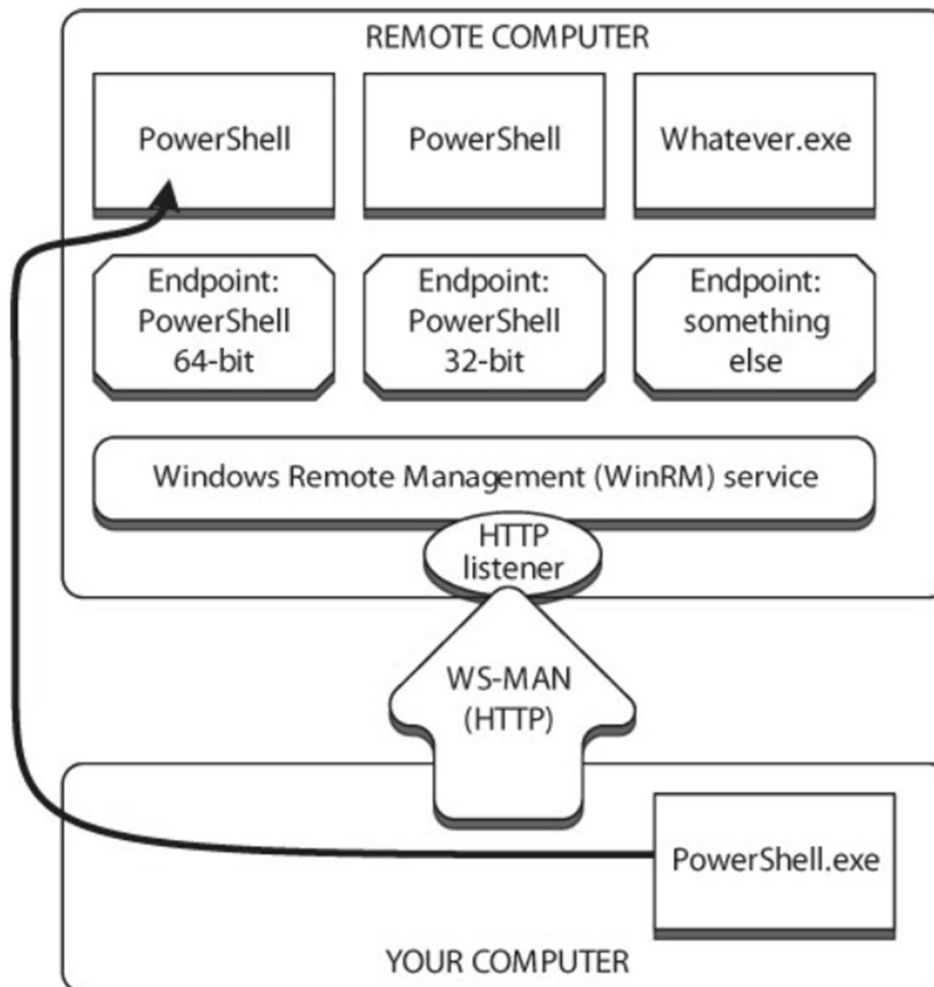
- In Powershell, when you run a remote command, it runs **on the remote system**.
- Only the **results** of the command are sent back to the computer that sent the command.
- Powershell does this using **Web Services for Management (WS-MAN)**.
 - Build off Windows Remote Management (WinRM)
 - Operates using HTTP/HTTPS (easier to route through firewalls).

WS-Man: What you need and how it works.

- Both computers (local and remote) must be running **PS v2.0** or greater.
- Must (should) be members of the same Forest (*if you don't want a bad time*).
(no seriously; I've had to get this to working before, it wasn't fun)
- **WS-MAN** (because of how WinRM works) allows you execute commands in the background, so users don't see what's happening.
- Can remote one-to-one (1:1) or one-to-many (1:N).
 - 1:1 remoting sends commands to a single remote computer
 - 1:N remoting sends commands to multiple remote computers

WinRM Overview

- Must be configured on the remote computers (the ones you want to connect to).
 - WinRM acts as a *dispatcher* on the remote computer.
 - When traffic enters the computers NIC, and is passed to the **application layer**, WinRM decides which application to send the data to.
 - Any **Recipient** must be a registered WinRM **endpoint**.
 - Each end point has an associated **listener**.
 - Each endpoint represents a different application (they are called **Sessions Configurations**).
- *Some applications can have multiple endpoints with differing permissions or rights.*



Serialization/Deserialization

- When data is sent back and forth, between two computers, the process of *serialization/deserialization* – takes place.
- Serialization converts the objects being sent to [XML](#).
- Deserialization converts the XML back into objects to be used by the local system.
- **Important: serialized data is *not live data*.**
- The collection that is being returned is a “Serialized” TypeName, which means the associated methods for that collection are not longer available (more on this later).

Remote Computer configuration

Each remote computer must have the following setup configured.

- Enable remoting by running the `Enable-PSRemoting` command (more on this soon)
- (If scripts will be run remotely) we must set the script execution policy accordingly.
- `Get-ExecutionPolicy` to see the current settings
- `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned`

Enabling PSRemoting

- There are *several* things that must be configured for a remote host to accept a RSRemote session.
- Good news! `Enable-PSRemoting` does it all for you.
make sure you run this as administrator
Network location (profile) must not be set to Public
 - Invokes `Set-WSManQuickConfig`
 - Starts the WinRM Service (if its not started)
 - Sets the WinRM service to “automatic” start-up type
 - Registers Powershell as an endpoint
 - Opens Windows Firewall rules required to allow traffic in.
 - TCP port 5985 (HTTP); TCP port 5986 (HTTPS)
- PSRemoting can also be configured using Group Policy
(We'll cover that in OSSE1091A)

Enter-PSSession/Exit-PSSession

1:1 Remoting

- Establish a session with the remote computer
 - `Enter-PSSession -ComputerName Server002-XXX`
 - Once connected the prompt will indicate that you are now running commands against the remote computer.
 - You can do anything on remote computer that you could do normally.
 - Because you're in a domain (and are using Domain Administrator) your permissions/rights on the remote computer will *also* be Domain Administrator (thanks Kerberos!).
(permissions, BTW, is one of the things that makes this so hard outside of a Domain environment)

Enter-PSSession/Exit-PSSession

1:1 Remoting

- Caveats:
 - Profiles you use won't load if you are remoting (this is why you should learn the default environment).
 - You are still restricted by the execution policy of the remote computer for scripts.

Set-Execution policy RemoteSigned
Get-Execution Policy

- To terminate your remote session:
Exit-PSSession (Exit)

Invoke-Command

- 1:N Remoting
- Using the `Invoke-Command` command, we can send a single command to *many* computers.
 - Each computer will independently respond.
 - The results are compiled into a single collection
 - Up to 32 computers can be controlled at a time (any extras will be queued)
- Syntax:

```
Invoke-Command `
    -ComputerName <stringArray> `
    -ScriptBlock {commandsToExecute}
```

Invoke-Command (cont'd)

- Example:
`Invoke-Command -ComputerName Server002-GGB,win001-GGB -ScriptBlock { Get-Process svchost | select -first 10 }`
- The `-command` parameter is actually an alias for `-scriptblock` (for when you're looking in help)
- `-FilePath` can also target a file, as well
- If you had a large script to run against many computers, you could type:
`Invoke-Command -computerName (Get-content .\Computers.txt) -FilePath.\script.ps1`

Invoke-Command (cont'd)

- An even BETTER example of the previous command is to run it using active directory commands *inline*.
(remember: the AD Module for PS is available for any computer running Remote Serve Administration Tools (RSAT))
- `Invoke-Command -ComputersName (Get-ADComputer -filter *
-SearchBase "cn=computers,dc=gber,dc=local" | select-
Object -ExpandProperty Name) -ScriptBlock { Get-
Process svchost | select -first 10 }`

Deserialized Objects

- When a command is run remotely, what comes back is **not the same** as if it were run locally.
- The data is converted to XML (Serialized) and then Converted *back* to a collection.
- If we look at the TypeName of Deserialized data,
 - The TypeName has changed
 - many of the methods are missing; meaning, the things we can “do” with this data are heavily reduced.

Deserialized Objects

- Invoke-Command`
-ComputerName Server002-GGB`
-ScriptBlock { Get-Process svchost | Where-Object
{ \$_.ID -lt 1000 } }

```
PS C:\Work> Get-Process svchost | Where-Object { $_.ID -lt 1000 }
TypeName: System.Diagnostics.Process

Name      MemberType Definition
-----
BeginErrorReadLine Method void BeginErrorReadLine()
BeginOutputReadLine Method void BeginOutputReadLine()
CancelErrorRead Method void CancelErrorRead()
CancelOutputRead Method void CancelOutputRead()
Close Method void Close()
CloseMainWindow Method bool CloseMainWindow()
Dispose Method void Dispose()
Equals Method bool Equals(System.Diagnostics.Process)
GetHashCode Method int GetHashCode()
GetType Method type GetType()
ToString Method string ToString()
Company NoteProperty string Company=Microsoft Corporation
CPU NoteProperty double CPU=0.2343
Description NoteProperty string Description=System.Diagnostics.Process
FileVersion NoteProperty string FileVersion=6.0.6002.18006
```



```
PS C:\Work> Invoke-Command -ComputerName Server002-GGB, WI
TypeName: Deserialized.System.Diagnostics.Process

Name      MemberType Definition
-----
GetType Method type GetType()
ToString Method string ToString()
Company NoteProperty string Company=Mi
CPU NoteProperty double CPU=0.2343
Description NoteProperty string Descriptio
FileVersion NoteProperty string FileVersio
```

Deserialized Objects

- `Invoke-Command -ComputerName Server002-GGB,win001-GGB -ScriptBlock { Get-Process notepad } | Stop-Process`

VS

- `Invoke-Command -ComputerName Server002-GGB,win001-GGB -ScriptBlock { Get-Process notepad | Stop-Process }`
- Example 1: The command will **fail**. The **deserialized data is not live data**. You cannot bring the data local, then run a method against it. **The local computer would try to stop the local Notepad.exe process.**
- Example 2: because the `GPS` and `Stop-Process` are constrained within `{ }` both commands will execute on the remote system and being back only the result of the second command.

Local vs Remote Processing

- `Invoke-Command -ComputerName Server002-GGB,win001-GGB -ScriptBlock {
Get-Process svchost | Where-Object {$_.ID -lt 1000} }`

VS

- `Invoke-Command -ComputerName Server002-GGB,win001-GGB -ScriptBlock {
Get-Process svchost } | Where-Object {$_.ID -lt 1000}`

What's the Difference?

- Example 1: data is **efficiently** filtered out on the remote computer before it is sent back to the local computer.
- Example 2: unfiltered data is sent back to the host. It is then filtered on the host. This increase the amount of data being retrieved and **slows the process down**.
- Remember: **filter left, filter close to the host**

The Rules...

1. Complete as much processing on the remote computer as possible.
2. Filter as much as possible on the remote computer so that, what becomes serialized will be small, and filtered.
3. A retrieved object that has been deserialized *lacks methods*. It is not live data.

Consider the command you are Invoking.

- If the command being invoked is exporting data or formatting data, it will work fine.
- If the command being invoked is an action, carefully consider the above two rules.

Things to remember.

- You must use the computers HOSTNAME when remoting (not DNS name, or IP). This can be modified with advanced configurations.
- PSRemoting works very well in a domain, but not in a workgroup. Reference `about_remote_troubleshooting` if you ever need to set this up.
- When running remote commands (not entering a PSSession) the remote computer launches Powershell, runs the command, then closes Powershell. The session doesn't stay open. Always run PS as admin.
- For firewalls:
 - Do not use a Public NIC profile
 - If you use 3rd Party Firewalls, the firewall rules won't be automatically created by `Enable-PSRemoting`

Variables

...it's a container!

- A Variable is container that stores data, so that data can be retrieved for later use. The data can be (not an exhaustive list):
 - XML
 - A Collection
 - An Object
 - A String or Integer/Float
- Variables in PS do not need to be declared (but they can be).
- And they can be reused! The `$var` can hold one thing for a moment and then something completely different another moment.

Objectification!

- **Everything** in PS is an object (even strings).
- Sometimes objects are referred to simply as **values** (because the value of an object is usually what we're interested in)
- While we are usually interested in the value, here's an example that shows we can still manipulate a string like an object.
- Consider:
 - `"Pickle" | GM`
 - `("pickle").GetType()`
 - Outputs a string object with many properties and methods.
 - `("Pickle").ToUpper()` ← Which we can then manipulate.

`"Pickle"`, *variablized*: Store the object to be used later...

- `$st = "Pickle"`
- `$st.ToUpper()`

How to Variablize an object

- We can store the objects values in variables.
 - Use `$<variableName>`
 - Followed by an *assignment operator* (=)
 - followed by whatever the `$var` is equal to
- Example:

```
PS C:\Work> $srv = "Server002-GGB"  
PS C:\Work> $Srv  
Server002-GGB
```

- It is important to note that `$` is *not* part of the variable name.
- The `$` tells the shell that “what’s next is a variable name”.

About Variables Names

- `$var` Names usually contain
 - Letters, and Numbers
 - And its common for them to contain an underscore (`_`)
- If a `$var` name contains a space, it must be curly braced { }
`${Var Name with Spaces}`
- **AVOID USING SPACES!** It just complicates things.
(**camelCase** instead)
 - `$varNameWithoutSpaces`
- `$vars` and their values are destroyed when the PS session closes
- Use *meaningful*, as-short-as-possible names for your `$vars`. Anyone who has to look at your code will thank you.

About Variables Names

- `$vars` in Powershell can be reused. You can write a value to it and then *redeclare* the value and it will change.

```
PS C:\Work> $srv = "Server002-GGB"  
PS C:\Work> $Srv  
Server002-GGB  
PS C:\Work> $srv = "Server001-GGB"  
PS C:\Work> $Srv  
Server001-GGB
```

Passing the contents of one Var to another

- We can reference `$vars` inside of other `$vars`!

```
PS C:\Work> $msg = "Wanna know a secret? $secret"
PS C:\Work> $msg
Wanna know a secret? Sam Brightens my day!
PS C:\Work> $secret
Sam Brightens my day!
PS C:\Work> $msg
Wanna know a secret? Sam Brightens my day!
```

You can quote me on this...

- Be careful when using quotes with variables.

```
PS C:\Work> $srv = "Server002-GGB"  
PS C:\Work> "Please log into $Srv"  
Please log into Server002-GGB  
PS C:\Work> 'Please log into $Srv'  
Please log into $Srv
```

- ‘ ‘ makes the string *literal*, ignoring any special characters as special
- “ ” defines a string but *observes* the special characters as special.

Another way to ignore variables in strings...

- ``` ← Backtick is the *escape character* of PS. The escape character **removes** the special meaning from anything that follows. Its also used to **format strings**.

```
PS C:\Work> "Please log into $srv"  
Please log into Server002-GGB  
PS C:\Work> "Please log into `$srv"  
Please log into $srv
```

- From the example above, we can see that, while Using “ ”, the addition of the ``` before `$srv` makes PS ignore the fact that this is a variable.

The Great Escape...character.

- ``n` (the escape character + the letter `n`) means “new line” in PS.
- When we put it between parts of a string, it will separate them into lines.

```
PS C:\Work> $thingsToDo = "Groceries`nDishes`nLaundry`nDog Walkies"
PS C:\Work> $thingsToDo
Groceries
Dishes
Laundry
Dog Walkies
```



The diagram illustrates how the escape sequence ``n` (backslash followed by 'n') in a PowerShell string is interpreted as a new line. It shows a PowerShell prompt where a variable `$thingsToDo` is assigned the string `"Groceries`nDishes`nLaundry`nDog Walkies"`. When the variable is printed, the output is displayed across four lines: `Groceries`, `Dishes`, `Laundry`, and `Dog Walkies`. Red arrows point from the ``n` sequences in the string to the line breaks in the output, and orange arrows point from the ``n` sequences in the string to the same ``n` sequences in the string representation shown below the output.

- There are quite a few of these escape characters. Reference `about_special_characters` help files for more details.

Common ways to...escape

Sequence	Description
-----	-----
<code>`0</code>	Null
<code>`a</code>	Alert
<code>`b</code>	Backspace
<code>`e</code>	Escape
<code>`f</code>	Form feed
<code>`n</code>	New line
<code>`r</code>	Carriage return
<code>`t</code>	Horizontal tab
<code>`u{x}</code>	Unicode escape sequence
<code>`v</code>	Vertical tab

Storing more than one object in a \$var

- Using a comma separated list (Array) of items.

```
PS C:\Work> $computers = 'Server001-GGB','Server002-GGB','Win001-GGB'
PS C:\Work> $computers
Server001-GGB
Server002-GGB
Win001-GGB
```

- Note: in this example the commas *cannot* be in the quotes or else our string will be a single item

```
PS C:\Work> $computers = 'Server001-GGB,Server002-GGB,Win001-GGB'
PS C:\Work> $computers
Server001-GGB,Server002-GGB,Win001-GGB
```

wrong

Accessing Properties/Methods of objects

- We can also manipulate an object stored in a `$var` by accessing its properties and methods (run a `GM` against a command to see what they are).

```
PS C:\Work> $computers | gm
    TypeName: System.String

Name      MemberType
-----
Clone     Method
CompareTo Method
Contains  Method
CopyTo    Method
EndsWith  Method
EnumerateRunes Method
Equals    Method
GetEnumerator Method
GetHashCode Method
GetPinnableReference Method
GetType   Method
GetTypeCode Method
```

```
PS C:\Work> $computers.count
3
PS C:\Work> $computers.ToUpper()
SERVER001-GGB
SERVER002-GGB
WIN001-GGB
PS C:\Work> $computers.GetType() | ft -AutoSize
IsPublic IsSerial Name      BaseType
-----
True     True     Object[] System.Array

PS C:\Work> $computers.Replace('GGB','XXX')
Server001-XXX
Server002-XXX
Win001-XXX
```

Referencing multiple objects from a \$var array

- We can then reference each item inside our \$var by using its index number.
 - [0], [1]...[x] ← Reference the index numbers forwards.
 - [-1], [-2] ← Reference the index numbers backwards.

```
PS C:\Work> $computers = 'Server001-GGB', 'Server002-GGB', 'Win001-GGB'
PS C:\Work> $computers[0]
Server001-GGB
PS C:\Work> $computers[1]
Server002-GGB
PS C:\Work> $computers[-1]
Win001-GGB
PS C:\Work> $computers[-2]
Server002-GGB
```

Working with multiple variable objects

- What if we are storing strings AND integers in the same variable? Or, we just want to do something with ONE of the variables and not the other? We can use the index numbers to specify which variable to modify.

```
PS C:\Work> $computers = 'Server001-GGB', 'Server002-GGB', 'Win001-GGB'
PS C:\Work> $computers[2].Replace('GGB', 'XXX')
Win001-XXX
```

- This is also useful if the `$var` is holding different object types (one **method** wont work on the other)

```
PS C:\Work> $multiObject = 100, 'one hundred'
PS C:\Work> $multiObject[0].Equals(99)
False
PS C:\Work> $multiObject[1].ToUpper()
ONE HUNDRED
```

Replacing the value of a variable

- **Note:** the previous examples (modifying the objects in a variable) **does not write the new object value back to the variable** (it simply creates a new, temporary value and outputs that to the screen (**its not stored anywhere**)).
- If we want to dynamically ***change*** the value held in the variable to the new value produced, we do it like this.

```
PS C:\Work> $computers = 'Server001-GGB', 'Server002-GGB', 'Win001-GGB'
PS C:\Work> $computers[2] = $computers[2].Replace('GGB', 'XXX')
PS C:\Work> $computers
Server001-GGB
Server002-GGB
Win001-XXX
```

Working with multiple \$var objects (cont'd)

- What if I have multiple objects in my \$var and I want to **run a method against each one after the other** (instead of a separate statement for each)?

```
PS C:\Work> $computers | ForEach-Object { $_.ToUpper() }  
SERVER001-GGB  
SERVER002-GGB  
WIN001-GGB
```

- For everything in \$computers, look at what's in the pipeline (\$_.)
- For each object in the pipeline, apply its toUpper method.
- If we want to write those values back to the variable:

```
PS C:\Work> $computers = $computers | % { $_.ToUpper() }  
PS C:\Work> $computers  
SERVER001-GGB  
SERVER002-GGB  
WIN001-GGB
```

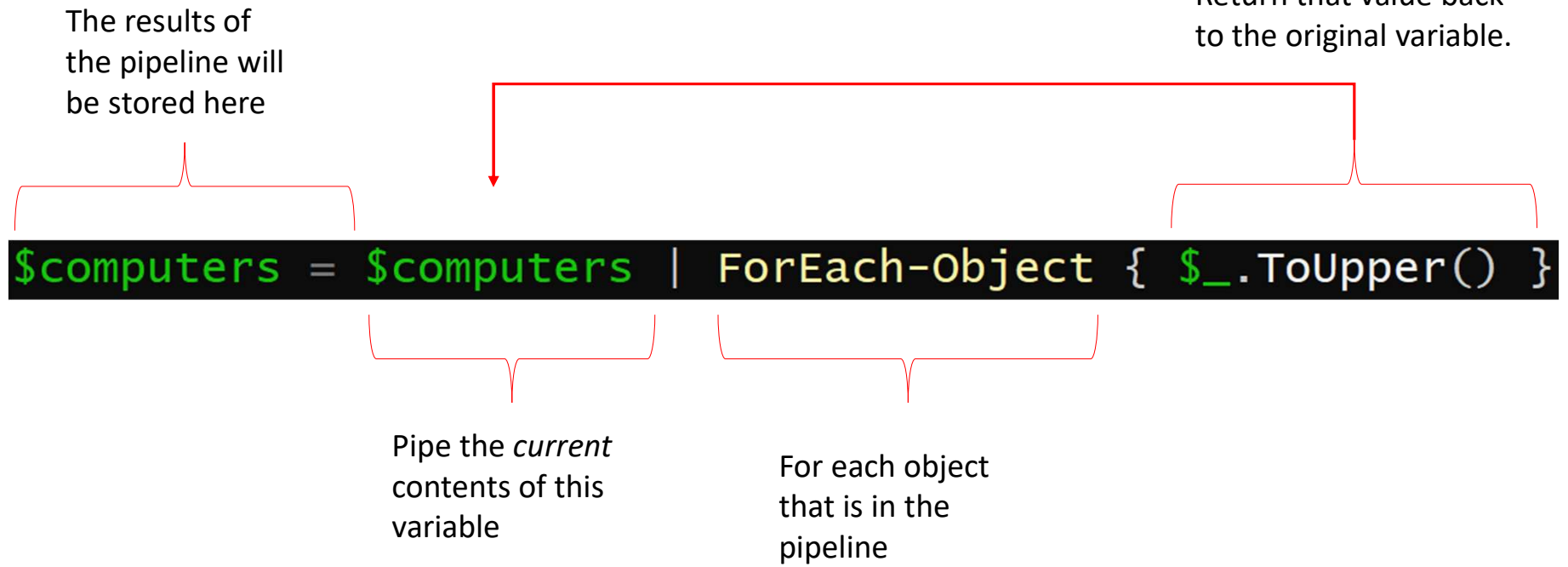
% is the alias for
For-EachObject



ForEach-Object Explained

The results of
the pipeline will
be stored here

Execute this method
(action) against it.
Return that value back
to the original variable.



```
$computers = $computers | ForEach-Object { $_.ToUpper() }
```

The diagram illustrates the PowerShell command `$computers = $computers | ForEach-Object { $_.ToUpper() }`. Red annotations explain its parts: a bracket above the first `$computers` points to the text 'The results of the pipeline will be stored here'; a bracket below the `$computers` and `=` points to 'Pipe the *current* contents of this variable'; a bracket below the `ForEach-Object` command points to 'For each object that is in the pipeline'; and a bracket above the script block `{ $_.ToUpper() }` points to 'Execute this method (action) against it. Return that value back to the original variable.' A red arrow also points from the `ForEach-Object` command to the first `$computers`.

Pipe the *current*
contents of this
variable

For each object
that is in the
pipeline

More than one way to skin a command...wut?

- You may have noticed that some of the ways we write commands overlap. This is because you're starting to understand how .net works.
- Consider these two commands. They are *identical* in function.

```
PS C:\Work> Get-Service | ForEach-Object { Write-Host $_.Name }  
PS C:\Work> Get-Service | Select-Object -ExpandProperty Name
```

- It really benefits you to pay attention to ***how .net works*** and not just learning PS syntax.

Using an object property in a string.

- Consider this example first. We are pulling the length property from our string.

```
PS C:\Work> $computers = 'Server001-GGB', 'Server002-GGB', 'Win001-GGB'
PS C:\Work> $compNameLen = $computers.Length
PS C:\Work> $compNameLen
3
```

- To use this info in-line in a string of text we must use a *subexpression* `$()`.

```
PS C:\Work> $compString = "The length of the string is $($computers.Length)"
PS C:\Work> $compString
The length of the string is 3
```

```
PS C:\Work> $services = Get-Service
PS C:\Work> $sName21 = "The name of the 21st service is $($services[21].name)"
PS C:\Work> $sName21
The name of the 21st service is CDPSvc
```

Declaring a `$vars` type

- PS works with data well enough to interpret how to use different data types in different situations, but sometimes you must *specify* the type of data in the `$var`.

```
PS C:\work> $num = Read-Host "Enter a number from 1 - 10"
Enter a number from 1 - 10: 10
PS C:\work> $num * 10
101010101010101010
```

- That's not what we meant PS! PS did its best and said: "Oh you want 10 "10's" (the *string* not the *number*), here's 10 10's.

:\

Declaring a \$vars type

```
PS C:\Work> $num | gm  
TypeName: System.Int32
```

- There's the problem. It's a string. We can fix this by *Declaring the \$var type* during its creation. Like this:

```
PS C:\Work> [int]$num = Read-Host "Enter a number from 1 - 10"  
Enter a number from 1 - 10: 10  
PS C:\Work> $num * 10  
100
```

```
PS C:\Work> $num | gm  
TypeName: System.Int32
```

Variable types in PS.

- `[int]` – Integers (non-decimal numbers)
- `[single]` or `[double]` – floating numbers (numbers w/e a decimal)
- `[string]` – A string of Characters
- `[char]` – Exactly one Character
- `[Xml]` – and XML specification document
- `[adsisearcher]` – AD Service Interface queries (Puts AD objects in the \$var)

Declaring your `$vars` prevents errors in scripts when the wrong type of Data is used.

```
PS C:\Work> [int]$num = Read-Host "Enter a number from 1 - 10"
Enter a number from 1 - 10: dog
MetadataError: Cannot convert value "dog" to type "System.Int32". Error: "The input string 'dog' was not in a correct format."
```

Variable Management commands.

- **Clear-Variable** – Deletes the value of a variable.
(you may use this one)
- **Get-Variable** – Gets the variables in the current console.
- **New-Variable** – Creates a new variable.
- **Remove-Variable** – Deletes a variable and its value.
(probably the only one you'll use)
- **Set-Variable** – Changes the value of a variable.

The Rules

- Variable names should be:
 - Simple
 - Descriptive
 - Single word, camel-cased names (`$dogFood`, not `${Dog Food}`)
- If the variable contains only one type of object, declare the type to prevent errors.
- *Get in the habit of declaring variable types.*